

Slurm para Usuários

Do primeiro sbatch a jobs MPI, GPU e containers

Albino Aveleda (albino@bino.eng.br)

Agenda

#	Capítulo
1	Conceitos: cluster, partições, jobs, steps e tasks
2	Explorando o cluster: `sinfo`, `squeue`, `scontrol`
3	Submetendo jobs: `sbatch`, `srun`, `salloc`
5	Exemplos de jobs: serial, array, MPI, GPU, Apptainer, Pyxis
7	Boas práticas e encerramento
L	Laboratório (sessão prática)

Capítulo 1

Conceitos fundamentais

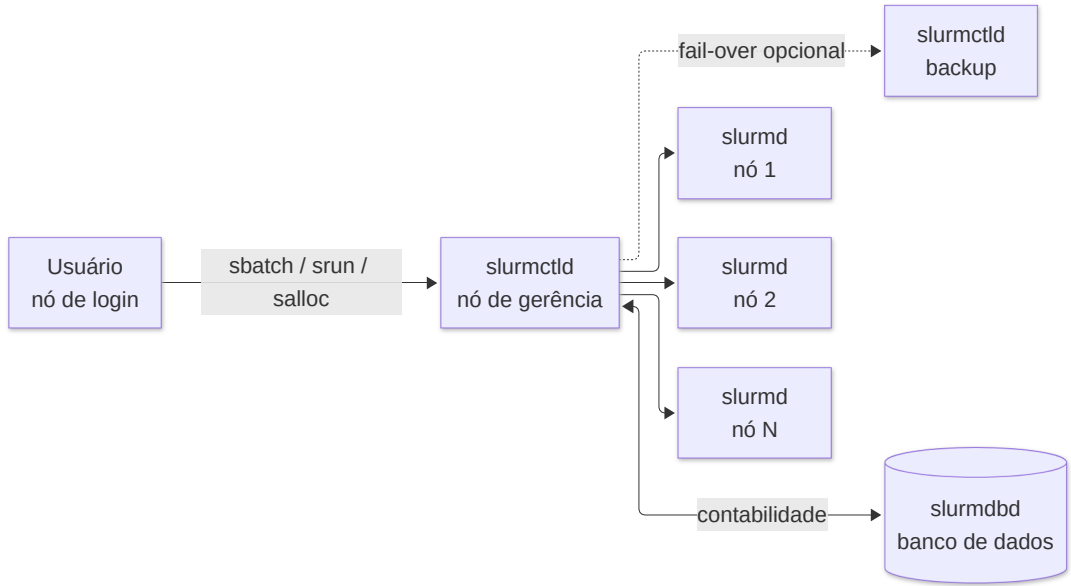
O que é o Slurm?

Gerenciador de cargas de trabalho (*workload manager*) para clusters Linux, com **três funções-chave**:

1. **Alocar** acesso exclusivo e/ou compartilhado a recursos (nós de computação) por um período de tempo
2. Fornecer um **framework para iniciar, executar e monitorar** trabalho (normalmente paralelo) nos nós alocados
3. **Arbitrar a disputa** por recursos, gerenciando uma fila de trabalhos pendentes

> "Slurm is an open source, fault-tolerant, and highly scalable cluster management and job scheduling system for large and small Linux clusters."

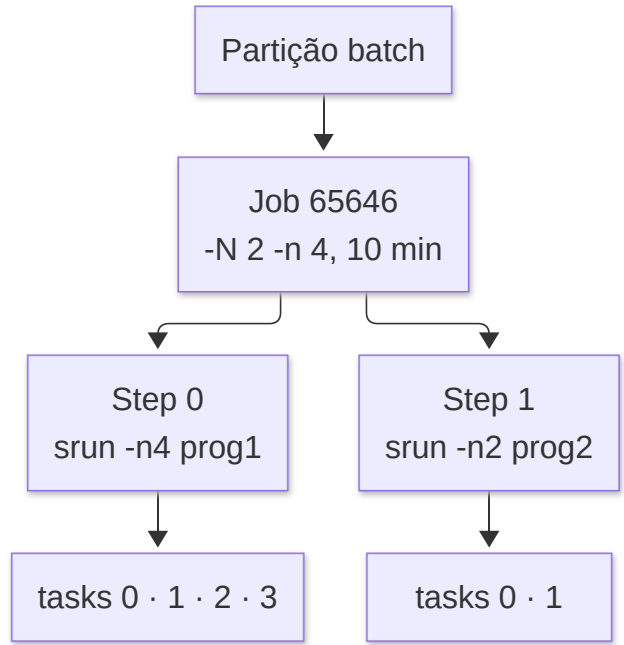
Arquitetura: quem faz o quê



- **slurmctld**: daemon central, decide quem roda onde
- **slurmd**: um por nó de computação, executa e acompanha as tarefas
- **slurmdbd**: guarda contabilidade (usada por `sacct`, fairshare, QOS)

As quatro entidades

- **Nó** (*node*): o recurso de computação
- **Partição** (*partition*): agrupamento lógico de nós, possivelmente sobreposto — funciona como **fila**
- **Job**: uma **alocação** de recursos atribuída a um usuário por um tempo determinado
- **Job step**: conjunto de **tasks** (possivelmente paralelas) dentro de um job



Do diagrama ao script

```
#!/bin/bash
# versão concorrente (& + wait)
#SBATCH -N 2       # 2 nós
#SBATCH -n 6       # 4 + 2 CPUs ao mesmo tempo
#SBATCH -t 10

srun -n4 ./prog1 &  # step 0 em segundo plano
srun -n2 ./prog2 &  # step 1 começa junto
wait                # espera os dois steps
```

```
JobID      NTasks
65646
65646.0         4   ← step 0
65646.1         2   ← step 1
```

- O `sbatch` **aloca** 2 nós e ≥ 4 CPUs (ex.: 2 por nó); o script roda no 1º nó
- **Step 0**: 4 processos espalhados nos 2 nós, usando toda a alocação
- Ao terminar, as CPUs ficam livres **dentro do job**
- **Step 1**: 2 processos, usando só **parte** da alocação
- Um step pode pedir **menos** que o job, nunca **mais**: o `srun` normalmente não põe mais de um processo por CPU
- Só com `-N 2` (sem `-n`): **1 task por nó**, e o `srun -n4` não caberia
- Sem `&`, o `srun` **espera** seu step terminar: o step 1 só começa depois do step 0

Partição = fila com regras

Cada partição tem restrições próprias:

- limite de **tamanho** de job (nós)
- limite de **tempo** (`MaxTime`)
- **quem** pode usar (grupos, contas, QOS)

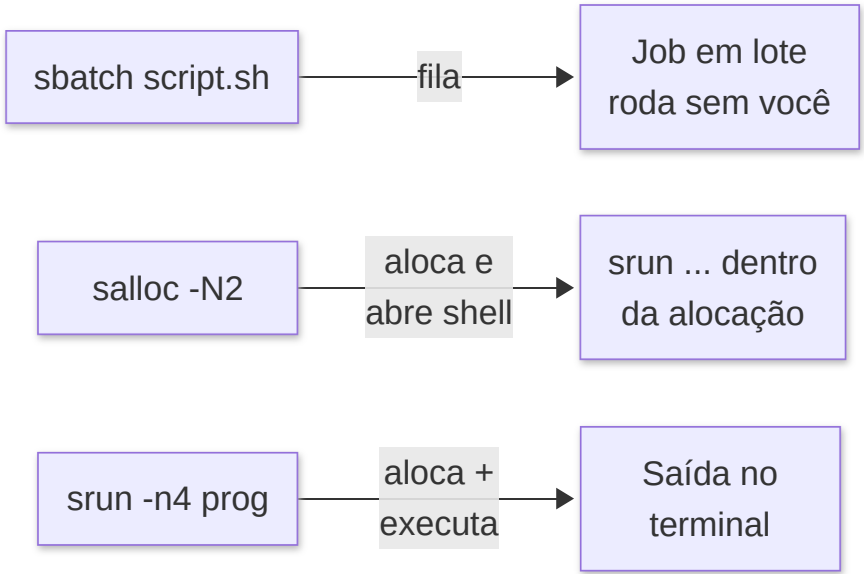
Para ver os limites do **seu** cluster: `scontrol show partition <nome>`

- `Default=YES` : partição usada quando o job não informa `-p`
- `DefaultTime` / `MaxTime` : tempo padrão e máximo dos jobs

Os comandos do usuário

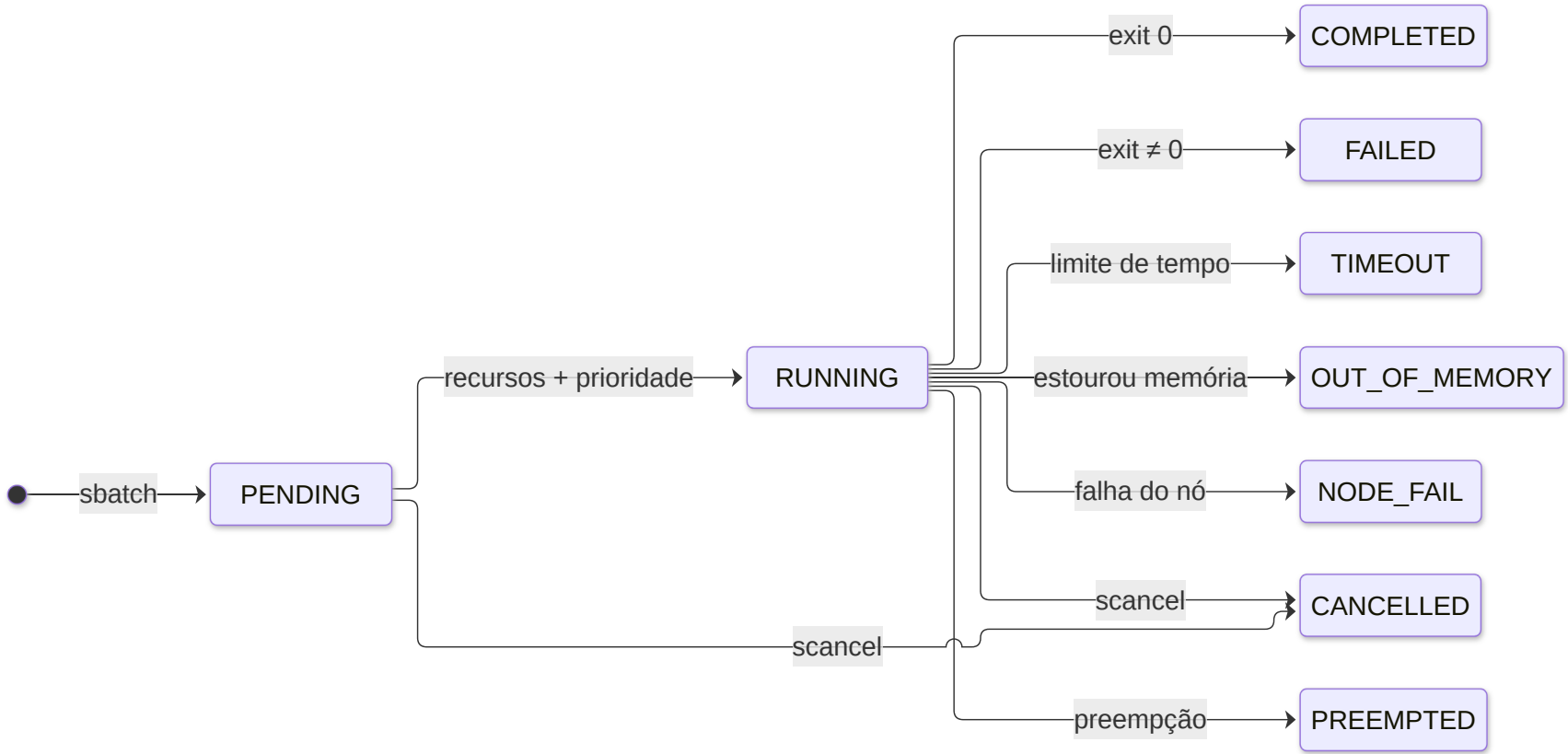
Comando	Para quê
`sbatch`	submeter um **script** para execução posterior
`srun`	submeter um job **ou** iniciar job steps em tempo real
`salloc`	obter uma alocação interativa (normalmente abre um shell)
`squeue`	estado dos jobs e steps
`sinfo`	estado das partições e nós
`scontrol`	ver (e às vezes alterar) o estado do Slurm
`scancel`	cancelar ou sinalizar jobs/steps
`sacct`	contabilidade de jobs ativos ou concluídos
`sstat`	uso de recursos de jobs **em execução**
`sprio` / `sshare`	prioridade e fairshare

Três modos de trabalho



- **Lote** (`sbatch`): o caso normal em produção
- **Interativo** (`salloc` + `srun`): depuração e testes
- **Direto** (`srun`): cria a alocação e lança o step de uma vez

Ciclo de vida de um job



Estados de job (tabela oficial)

Estado	Significado
`PENDING` (`PD`)	na fila; normalmente com um *reason code* explicando por quê
`RUNNING` (`R`)	recursos alocados e executando
`COMPLETED` (`CD`)	terminou com exit code zero em todos os nós
`FAILED` (`F`)	terminou com exit code diferente de zero ou outra falha
`TIMEOUT` (`TO`)	atingiu o limite de tempo
`OUT_OF_MEMORY` (`OOM`)	erro de falta de memória
`CANCELLED` (`CA`)	cancelado pelo usuário ou administrador
`NODE_FAIL` (`NF`)	falha de nó
`PREEMPTED` (`PR`)	interrompido por preempção
`SUSPENDED` (`S`)	recursos alocados, execução suspensa

Flags adicionais comuns: `COMPLETING` (limpeza/epilog), `CONFIGURING` (nós iniciando), `REQUEUED`

Steps são exclusivos por padrão ≥ 20.11

Dentro de uma alocação, cada `srun` cria um **step** que, por padrão, **não compartilha** recursos com outros steps em paralelo.

```
salloc -n1
date +%T; srun -n1 --exclusive -l sleep 3 &
srun -n1 --exclusive -l date +%T &
# srun: Job 9553 step creation temporarily disabled, retrying (Requested nodes are busy)
```

Com `--overlap` os steps compartilham CPUs, memória e GRES e começam juntos:

```
date +%T; srun -n1 --overlap -l sleep 3 &
srun -n1 --overlap -l date +%T &
```

O mesmo em um batch: `&` × reserva

```
#!/bin/bash
#SBATCH -n 1          # job com 1 CPU
#SBATCH -t 5
date +%T
# step 0: pega a única CPU e roda
srun -n1 -l sleep 10 &
# step 1: srun lançado já, mas o step
# fica pendente até a CPU liberar
srun -n1 -l date +%T &
# step 2: --overlap divide a CPU, roda já
srun -n1 --overlap -l date +%T &
# & só faz o script não esperar o srun
wait
```

Linha do tempo:

```
t=0s   step 0 começa (sleep 10)
t=0s   step 2 começa e imprime a hora
t=0s   step 1 aguardando a CPU...
t=10s  step 0 termina e libera a CPU
t=10s  step 1 começa e imprime a hora
```

- `&` : o **script** não espera o `srun` , os 3 são **lançados** juntos
- **Reserva exclusiva** (padrão): o **step** só roda se houver CPU livre
- `--overlap` : o step **divide** a CPU com os outros, **não** adiciona CPU; bom para steps leves e monitoramento (`top` / `nvidia-smi`)

Capítulo 2

Explorando o cluster

`sinfo` : partições e nós

```
$ sinfo
PARTITION AVAIL TIMELIMIT NODES STATE  NODELIST
batch     up     infinite     2 alloc  adev[8-9]
batch     up     infinite     6 idle   adev[10-15]
debug*    up        30:00     8 idle   adev[0-7]
```

- O `*` indica a **partição padrão**
- Uma partição pode aparecer em **várias linhas** — uma por estado de nó
- `NODELIST` usa **expressões de faixa**: `adev[10-15]`

`squeue` : a fila

```
$ squeue
JOBID PARTITION  NAME  USER ST       TIME NODES NODELIST(REASON)
65645     batch  phys  anna  R 1-02:13:45     4 adev[1-4]
65646     batch  chem  mike  R      24:19     2 adev[7-8]
65647     batch   bio  joan  R       0:09     1 adev14
65648     batch  math  phil PD       0:00     6 (Resources)
```

- `ST` : `R` = running, `PD` = pending
- `TIME` : formato `dias-horas:minutos:segundos`
- `NODELIST(REASON)` : onde está rodando **ou** por que ainda espera
- Motivos mais comuns: **Resources** (aguardando recursos) e **Priority** (atrás de job mais prioritário)
- Por padrão: jobs em execução por prioridade, depois pendentes por prioridade

`squeue` : filtros do dia a dia

```
squeue -u $USER
squeue -u $USER -t running
squeue -u $USER -t pending
squeue --jobs 12345,12346,12347,12348
squeue -s -p debug -S u
squeue --priority --sort=-p,i --states=PD
```

- `-t` filtra por estado; `-s` mostra **steps** em vez de jobs
- `-S u` ordena por usuário
- A última linha lista os pendentes **na ordem em que o Slurm vai considerá-los**

`squeue` : formato personalizado

O formato padrão pode ser definido via variável de ambiente:

```
export SQUEUE_FORMAT="%.18i %.9P %.8j %.8u %.10T %.9Q %.10M %.9l %.6D %.6C %R"
```

```
$ squeue
             JOBID PARTITION     NAME     USER      STATE  PRIORITY       TIME TIME_LIMI  NODES   CPUS NODELIST(REASON)
             65645     batch     phys     anna    RUNNING      4521 1-02:13:45 2-00:00:0      4    128 adev[1-4]
             65646     batch     chem     mike    RUNNING      3870      24:19   4:00:00      2     64 adev[7-8]
             65647     batch      bio     joan    RUNNING      3122       0:09   1:00:00      1      8 adev14
             65648     batch     math     phil    PENDING      5012       0:00  12:00:00      6    192 (Resources)
```

Ou o formato longo, por nomes de campos:

```
export SQUEUE_FORMAT2="JobID:8,Partition:11,QOS:7,Name:10 ,UserName:9,Account:9,State:8,PriorityLong:9,ReasonList: ,Time
```

E para mudar o formato de data/hora em todos os comandos:

```
export SLURM_TIME_FORMAT="%a %T"
```

`squeue` e arrays / steps

```
squeue                      # agrupa tarefas pendentes: 1080_[5-1024]
squeue -r                   # uma linha por elemento do array
squeue -j 1080_2,1080_3     # elementos específicos
squeue --steps 65552.1      # um step específico
```

```
$ squeue --steps 65552.1
 STEPID     NAME PARTITION    USER    TIME  NODELIST
65552.1    test2     debug   alice   12:49  dev[1-4]
```

`scontrol show`: o raio-X

```
scontrol show partition
scontrol show node adev1
scontrol show hostnames tux[1-3]
scontrol show job 71701
```

```
JobId=71701 Name=hostname
   UserId=da(1000) GroupId=da(1000)
   Priority=66264 Account=none QOS=normal WCKey=*123
   JobState=COMPLETED Reason=None Dependency=(null)
   TimeLimit=UNLIMITED Requeue=1 Restarts=0 BatchFlag=0 ExitCode=0:0
   NumNodes=1 NumCPUs=10 CPUs/Task=2 ReqS:C:T=1:1:1
```

`scontrol show partition`: ler os limites

```
PartitionName=debug
   AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
   AllocNodes=ALL Default=YES QoS=N/A
   DefaultTime=NONE DisableRootJobs=NO Exclusive=NO GraceTime=0 Hidden=NO
   MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED
   Nodes=snowflake[0-48]
   DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
   TRES=cpu=588,mem=1204224M,node=49,billing=49
```

- `Default=YES`: partição usada quando você não informa `-p`
- `MaxTime`, `MaxNodes`: o que você pode pedir
- `DefMemPerNode` / `MaxMemPerNode`: memória padrão e máxima
- `QoS=`: QOS de partição (Capítulo 4)

Capítulo 3

Submetendo jobs: `sbatch`, `srun`, `salloc`

`sbatch` em uma frase

> Submete um **script** ao Slurm e **retorna imediatamente** com um Job ID. O script pode esperar na fila antes de receber recursos.

- Quando a alocação é concedida, o Slurm roda **uma cópia** do script em um dos nós (normalmente o primeiro)
- Saída padrão e erro vão para `slurm-%j.out` (gerado no **primeiro nó** da alocação)
- Fora o próprio script, **o Slurm não move arquivos do usuário** — use sistema de arquivos compartilhado

O primeiro script

```
#!/bin/sh
#SBATCH --time=1
srun hostname |sort
```

```
$ sbatch -N4 myscript
$ cat slurm-65537.out
host1
host2
host3
host4
```

Também é possível passar o script pela **entrada padrão**:

```
$ sbatch -N4 <<EOF
> #!/bin/sh
> srun hostname |sort
> EOF
sbatch: Submitted batch job 65541
```

Regras das diretivas `#SBATCH`

- Cada linha `#SBATCH` aceita **qualquer opção** da linha de comando do `sbatch`
- São lidas **pelo Slurm**, não pelo shell: `$VARIAVEL` vira **texto literal**
- O processamento **para na primeira linha que não é comentário nem espaço**
- Precedência: **linha de comando** > **variáveis de ambiente** (`SBATCH_*`) > **script**

```
#!/bin/bash
#SBATCH --job-name=teste
echo "início"                  # <- primeira linha de comando
#SBATCH --time=10              # IGNORADA!
#SBATCH --output=$HOME/x.out   # mesmo se lida, $HOME não seria expandido
```

Script completo: exemplo

```
#!/bin/bash
#SBATCH -J meu_job
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 4
#SBATCH --mem=8G            # sem forma curta
#SBATCH -p gpuq
#SBATCH -t 1-02:00:00
#SBATCH -o %x_%j.log
#SBATCH -G h200:2
...
```

> Especifique **nós e tasks** para que o job ocupe nós inteiros.

Opções essenciais

Opção longa	Curta	O que pede
`--job-name=<nome>`	`-J`	nome do job (padrão: nome do script)
`--partition=<p>`	`-p`	partição; aceita lista separada por vírgula
`--account=<conta>`	`-A`	conta a ser cobrada
`--time=<t>`	`-t`	limite de tempo total
`--nodes=<min[-max]>`	`-N`	número de nós
`--ntasks=<n>`	`-n`	número máximo de tasks que os steps vão lançar
`--cpus-per-task=<c>`	`-c`	CPUs por task (threads)
`--mem=<tam>`		memória **por nó**
`--mem-per-cpu=<tam>`		memória **por CPU alocada**
`--output` / `--error`	`-o` / `-e`	arquivos de saída/erro
`--chdir=<dir>`	`-D`	diretório de trabalho do script
`--constraint=<lista>`	`-C`	*features* exigidas dos nós

Tempo: formatos aceitos

```
#SBATCH --time=30            # minutos
#SBATCH --time=30:00         # minutos:segundos
#SBATCH --time=02:00:00      # horas:minutos:segundos
#SBATCH --time=1-12          # dias-horas
#SBATCH --time=1-12:30       # dias-horas:minutos
#SBATCH --time=1-02:00:00    # dias-horas:minutos:segundos
```

- Resolução de **1 minuto**; segundos são arredondados para cima
- Pedir mais que o limite da partição → job fica **PENDING** (possivelmente para sempre)
- Sem `--time` : vale o **tempo padrão da partição**
- No fim do tempo: **SIGTERM**, depois **SIGKILL** (intervalo = `KillWait`)

Padrões de nome de arquivo

```
#SBATCH --output=%x-%j.out       # nome-do-job + jobid
#SBATCH --error=%x-%j.err
```

Código	Substituído por	Código	Substituído por
`%j`	Job ID	`%A`	Job ID "mestre" do array
`%x`	nome do job	`%a`	índice do array
`%u`	usuário	`%J`	`jobid.stepid`
`%N`	hostname curto (1 arquivo por nó)	`%s`	step id
`%t`	rank da task (1 arquivo por task)	`%4j`	Job ID com zeros à esquerda

- `job%4j.out` → `job0128.out`
- `job%2j-%2t.out` → `job128-00.out, job128-01.out, ...`

Variáveis de ambiente dentro do job

```
#!/bin/bash
echo "Job ID:        $SLURM_JOB_ID"
echo "Nome:          $SLURM_JOB_NAME"
echo "Nós:           $SLURM_JOB_NODELIST"
echo "Tasks:         $SLURM_NTASKS"
echo "CPUs/task:     $SLURM_CPUS_PER_TASK"
echo "Submetido de:  $SLURM_SUBMIT_DIR"
```

- `SLURM_CPUS_PER_TASK` só é definida se você usou `--cpus-per-task`
- `SLURM_NTASKS` também é **entrada** do `srun`: define o `--ntasks` dos steps
- Variáveis `SLURM_*` são **sempre** propagadas, mesmo com `--export=NONE`

`--export` : o ambiente do job

```
sbatch --export=ALL job.sh              # padrão: propaga todo o ambiente
sbatch --export=NONE job.sh             # ambiente limpo (SLURM_* continuam)
sbatch --export=ALL,ENTRADA=dados.txt job.sh
```

- Padrão: `ALL` — o job herda o ambiente **do momento da submissão** (módulos carregados inclusive)
- `NONE` : só `SLURM_*` (e opções SPANK) vêm da submissão; o Slurm tenta carregar o **ambiente de login** no nó, como `--get-user-env` — carregue os módulos **dentro** do script
- Com `NONE` , os steps herdam essa configuração; para evitar, use `export SLURM_EXPORT_ENV=ALL` no script
- Dica: **não** altere manualmente `LD_LIBRARY_PATH` ; use módulos

```
#!/bin/bash
#SBATCH --export=NONE
module purge                # parte de um ambiente conhecido
module load gcc/14.2.0      # versão explícita
srun ./meu_programa
```

`srun` : lançar tasks

```
$ srun -n4 -l /bin/hostname
0: adev3
1: adev3
2: adev3
3: adev3
```

- Fora de uma alocação: cria a alocação e o step
- Dentro de `sbatch` / `salloc` : cria apenas um step na alocação existente
- `-l` prefixa cada linha com o número da task
- Padrão: uma task por nó (`-N`) ou uma CPU por task (`-n`)

Componentes heterogêneos com `:`

```
$ srun -n1 -c8 --mem-per-cpu=2gb server : -n16 --mem-per-cpu=1gb client
```

Em script, separando componentes com `#SBATCH hetjob`:

```
#!/bin/bash
#SBATCH -n 1 -c 8 --mem-per-cpu=2g    # 1º bloco (componente 0): server
#SBATCH hetjob                        # separa os blocos
#SBATCH -n 16 --mem-per-cpu=1g        # 2º bloco (componente 1): clients
srun --het-group=0 server : --het-group=1 client
```

O script roda no **primeiro nó do primeiro componente** (1 task, 8 CPUs, 16 GiB). Sem `--het-group`, o `srun` usaria **só o componente 0**.

`salloc` : sessão interativa

```
$ salloc -N5 srun -n10 myprogram
```

Recomendado **a partir da 20.11**, se o seu cluster tiver o *interactive step* habilitado:

```
$ salloc -N1 -n4 -t 30
# o shell abre direto em um nó da alocação
$ srun -n4 ./teste
$ exit
```

Caso contrário, o shell fica no **nó de login** (o `srun` ainda roda nos nós alocados). Para entrar no nó:

```
$ salloc -N1 -n4 -t 30
$ ssh $SLURM_NODELIST    # se o site permitir ssh (pam_slurm_adopt)
```

Dependências entre jobs

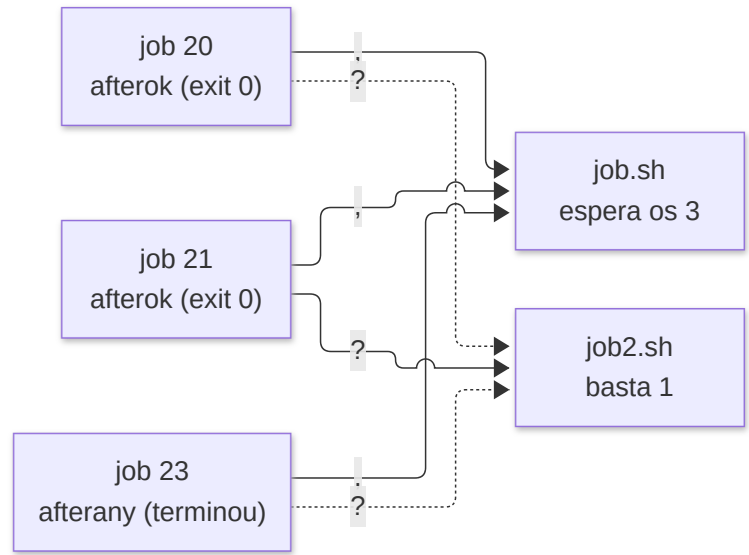
```
JOB1=$(sbatch --parsable pre.sh)
JOB2=$(sbatch --parsable -d afterok:$JOB1 calc.sh)
sbatch -d afterany:$JOB2 limpeza.sh
sbatch -d singleton -J banco job.sh
```

Tipo	Libera quando…
`after:id[+min]`	o job **iniciou** ou foi cancelado (+ minutos)
`afterany:id`	o job **terminou** (qualquer estado)
`afterok:id`	terminou com **exit code 0**
`afternotok:id`	terminou em estado de **falha**
`aftercorr:id`	tarefa de array **correspondente** completou
`singleton`	não há outro job **com o mesmo nome e usuário** rodando

Combinando dependências

```
sbatch -d afterok:20:21,afterany:23 job.sh   # E: todas as condições
sbatch -d afterok:20:21?afterany:23 job2.sh  # OU: qualquer condição
```

- `,` = **todas** devem ser satisfeitas
- `?` = **qualquer uma** basta
- Só um tipo de separador por vez
- Podem ser alteradas depois com `scontrol update`
- Dependência impossível → reason **DependencyNeverSatisfied**



Avisos antes do fim: `--signal`

```
#!/bin/bash
#SBATCH --time=01:00:00
#SBATCH --signal=B:USR1@300
trap 'echo "salvando checkpoint"; ./salvar_estado' USR1
./simulacao &
wait
```

- Envia `sig_num` quando faltam `sig_time` segundos para o fim
- Pode chegar até **60 s antes** do pedido (resolução do Slurm)
- `B:` sinaliza **apenas o shell do batch** (por padrão, sinaliza os steps, não o shell)
- Sem `@tempo`, o padrão é **60 s**

Capítulo 5

Exemplos de jobs

Serial · Array · MPI · GPU · Apptainer · Pyxis

Mapa dos exemplos

Tipo	Quando usar	Pedido típico
Serial	1 processo, 1 CPU	`-n 1`
Multithread	OpenMP / threads	`-n 1 -c N`
Array	mesma tarefa, muitos parâmetros	`--array=0-99%10`
MPI	processos distribuídos entre nós	`-N 2 -n 48`
GPU	CUDA / ROCm	`--gres=gpu:1` ou `--gpus=…`
Apptainer	software empacotado em `.sif`	`apptainer exec …`
Pyxis	imagens OCI/Docker via `srun`	`--container-image=…`

5.1 Jobs seriais

Serial mínimo

```
#!/bin/bash
#SBATCH --job-name=serial
#SBATCH --partition=batch
#SBATCH --ntasks=1
#SBATCH --time=00:10:00
#SBATCH --output=%x-%j.out
module load c/14.2.0
./meu_programa entrada.dat
```

- Muitos sites restringem jobs pequenos a **partições específicas** — confira as regras do seu cluster
- Sem `srun`: o programa roda **no batch step**, sem step separado na contabilidade

Serial: quatro formas equivalentes

Script | Script + srun | --wrap | Interativo

```
#!/bin/bash
#SBATCH -n 1
#SBATCH -t 10
./meu_programa
```

Usar `srun` dentro do script dá **contabilidade por step** (`sstat` , `sacct`) — vale para seriais também.

Serial multithread (OpenMP)

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --time=01:00:00
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun ./programa_openmp
```

- **Uma** task com **8 CPUs**
- `OMP_NUM_THREADS` acompanha o que foi pedido (`SLURM_CPUS_PER_TASK` só existe se `-c` foi usado)
- Atenção ao *binding*: threads herdam a máscara de CPU do processo pai

5.2 Job arrays

Variáveis de ambiente do array

Variável	Valor
`SLURM_ARRAY_JOB_ID`	Job ID do primeiro job do array
`SLURM_ARRAY_TASK_ID`	índice deste elemento
`SLURM_ARRAY_TASK_COUNT`	número de elementos
`SLURM_ARRAY_TASK_MAX`	maior índice
`SLURM_ARRAY_TASK_MIN`	menor índice
`SLURM_ARRAY_TASK_STEP`	passo dos índices

Exemplo oficial: `sbatch --array=1-3 -N1 tmp` gera os jobs `36`, `37`, `38` com:

```
SLURM_JOB_ID=36  SLURM_ARRAY_JOB_ID=36  SLURM_ARRAY_TASK_ID=3
SLURM_JOB_ID=37  SLURM_ARRAY_JOB_ID=36  SLURM_ARRAY_TASK_ID=1
SLURM_JOB_ID=38  SLURM_ARRAY_JOB_ID=36  SLURM_ARRAY_TASK_ID=2
```

⚠️ A ordem JOB_ID ↔ TASK_ID **não é garantida**.

Array: um arquivo de entrada por elemento

```
#!/bin/bash
#SBATCH --job-name=varredura
#SBATCH --array=1-100%10
#SBATCH --output=logs/%x_%A_%a.out
#SBATCH -n 1 -t 00:20:00
ENTRADA=$(sed -n "${SLURM_ARRAY_TASK_ID}p" lista_de_entradas.txt)
srun ./processa "$ENTRADA"
```

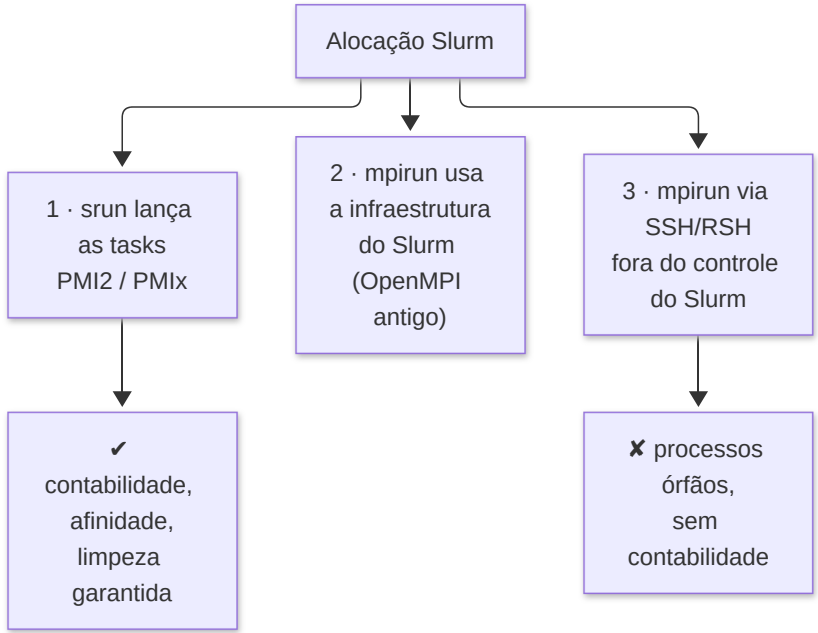
- `%A` = Job ID do array, `%a` = índice → um log por elemento
- Padrão sem `-o`: `slurm-%A_%a.out`
- A **linha N** da lista vira a entrada do elemento N

Gerenciando arrays

```
scancel 20_[1-3]                          # elementos 1 a 3
scancel 20_4 20_5                         # elementos 4 e 5
scancel 20                                # array inteiro
squeue -r                                 # uma linha por elemento
scontrol update JobID=21845_2 name=arturo # altera um elemento
scontrol suspend 21845                    # suspende o array inteiro
scontrol update JobId=20 ArrayTaskThrottle=8  # muda o limite %
```

5.3 Jobs MPI

Três maneiras de lançar MPI



Recomendado: **modo 1** — `srun` direto, com **PMIx** ou **PMI-2**.

Quais PMIs o seu Slurm tem?

```
$ srun --mpi=list
MPI plugin types are...
  cray_shasta
  none
  pmi2
  pmix
specific pmix plugin versions available: pmix_v3,pmix_v4
```

- `pmix` = a **maior** versão PMIx instalada; pode-se pedir uma específica (`pmix_v4`)
- Sem `--mpi` , vale o **padrão do site**
- Aplicação e Slurm devem usar a **mesma versão** de PMIx

MPI: exemplo completo

```
#!/bin/bash
#SBATCH --partition=batch
#SBATCH -N 2      # 2 nós
#SBATCH -n 48     # 24 processos MPI por nó, 48 tasks no total
#SBATCH --time=1-02:00:00
#SBATCH --output=mpi_job_slurm_output.log
#SBATCH --error=mpi_job_slurm_errors.log
module load openmpi4/gcc/4.1.8
srun ./meu_app_mpi
```

- **2 nós inteiros** × 24 núcleos: nós e tasks especificados juntos
- Carregue o **mesmo módulo MPI** usado para compilar
- Não misture toolchains de versões diferentes

`srun` × `mpirun`

	`srun`	`mpirun` **(Hydra, SSH…)**
Rastreamento de processos	✔️	depende
Contabilidade por step	✔️	parcial / ✘
Afinidade de tasks	✔️	configurar à parte
Suspend/resume	✔️	✘
Limpeza ao cancelar	✔️	pode deixar órfãos

Se precisar de `mpirun`, use a versão **integrada ao Slurm** e **não** passe `-machinefile` manual, a menos que a sua MPI exija.

Híbrido MPI + OpenMP

```
#!/bin/bash
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=6
#SBATCH --time=02:00:00
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun --mpi=pmix ./app_hibrido
```

- 2 nós × 4 ranks × 6 threads = **48 CPUs**
- Cada rank fica confinado às suas 6 CPUs (cgroup + afinidade)
- Ajuste `--ntasks-per-node` × `--cpus-per-task` ao número de núcleos do nó

Apptainer + MPI

```
#!/bin/bash
#SBATCH -N 2-4
#SBATCH -n 8
module load openmpi4/gcc/4.1.8
mpirun -n $SLURM_NTASKS apptainer exec mpi_hello.sif /opt/mpi_hello
```

- Modelo "**híbrido**" do Apptainer (não confundir com MPI + OpenMP): MPI do host lança os processos, cada um dentro do container
- A MPI de dentro do container precisa ser **compatível** com a do host

5.4 Jobs com GPU

Pedindo GPUs: as opções

Opção	Significado
`--gres=gpu:2`	2 GPUs **por nó** (forma clássica)
`--gres=gpu:h100:2`	2 GPUs **do tipo** `h100` por nó
`--gpus=N` / `-G`	GPUs **no job inteiro** (aceita tipo: `--gpus=h200:4`)
`--gpus-per-node=N`	por nó — equivalente a `--gres` para GPUs
`--gpus-per-task=N`	por task (exige número de tasks)
`--gpus-per-socket=N`	por socket
`--cpus-per-gpu=N`	CPUs por GPU
`--mem-per-gpu=X`	memória por GPU

GPU: CPUs proporcionais às GPUs

```
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH -N 1
#SBATCH -n 8
#SBATCH --gres=gpu:1
#SBATCH --time=02:00:00
srun ./treino_cuda
```

- Regra comum: dividir as CPUs do nó **proporcionalmente** às GPUs
- Ex.: nó com **10 GPUs** e **80 CPUs** → **8 CPUs por GPU**
- Ex.: nó com **4 GPUs** e **128 CPUs** → **32 CPUs por GPU**
- Muitos sites **proíbem** jobs só de CPU nas partições de GPU

5.6 Containers com Pyxis + Enroot

O que é o Pyxis

- **Plugin SPANK** (*Slurm Plug-in Architecture for Node and job (K)control*) do Slurm, mantido pela NVIDIA
- Permite a usuários **sem privilégio** rodar tarefas em containers via `srun`
- Usa o **Enroot** (containers em *user namespace*) por baixo
- Novas opções aparecem **direto** no `srun` (e no `sbatch`): `--container-image`, `--container-mounts`...

```
srun --help | grep container    # seu cluster tem Pyxis?
```

Pyxis: primeiros passos

```
$ srun grep PRETTY /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"

$ srun --container-image=almalinux:9 grep PRETTY /etc/os-release
pyxis: importing docker image: almalinux:9
pyxis: imported docker image: almalinux:9
PRETTY_NAME="AlmaLinux 9.7 (Moss Jungle Cat)"
```

Formatos de imagem aceitos:

```
srun --container-image=rocky grep PRETTY /etc/os-release       # Docker Hub
srun --container-image nvcr.io/nvidia/pytorch:26.03-py3 ...    # outro registry
srun --container-image ~/ubuntu.sqsh ...                       # arquivo squashfs local
```

Pyxis: montando dados

```
srun --container-image ubuntu --container-mounts /mnt:/data ls /data
srun --container-image ubuntu:24.04 --container-mounts tmpfs:/tmp df -h /tmp
srun --container-image ubuntu --container-mounts ./config:/root/config:ro,/mnt:/data ls /data /root/config
```

- `SRC:DST[:FLAGS][,SRC:DST...]`: várias montagens separadas por **vírgula**; vários flags separados por `+` (ex.: `ro+rprivate`)
- `:ro` = somente leitura
- `tmpfs:/tmp` = `/tmp` em memória dentro do container
- `--container-workdir`, `--container-mount-home`, `--container-writable` completam o kit

Pyxis em batch

```
$ sbatch --wait -o slurm.out <<EOF
#!/bin/bash
#SBATCH --container-image=nvcr.io/nvidia/pytorch:26.09-py3
python -c 'import torch ; print(torch.__version__)'
EOF
```

- Com `--container-image` no `#SBATCH`, **o script inteiro** roda dentro do container

Pyxis: reaproveitando containers

```
srun --container-image=ubuntu:20.04 which file
srun --container-image=ubuntu:20.04 --container-name=myubuntu --container-writable \
     --container-remap-root sh -c 'apt update && apt install -y file'
srun --container-name=myubuntu which file
```

Salvar um container pronto como **squashfs** e usar em 4 nós:

```
srun --ntasks=1 --container-image=nvcr.io/nvidia/pytorch:26.09-py3 --container-save /work/users/tng/pytorch.sqsh true
srun --nodes=4 --container-image=/work/users/tng/pytorch.sqsh python train.py
```

- **Cenário A: imagem do registry** em 4 nós: **4 downloads** + **4x extração**, cada nó importa a imagem no seu disco local
- **Cenário B:** `.sqsh` **compartilhado** (`/work`): importada **uma vez**, os 4 nós leem o mesmo arquivo (recomendado)

Entrar no container de um job **em execução**:

```
srun --overlap --jobid=432788 --container-name=myapp findmnt /data
```

Pyxis: job GPU completo

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --partition=gpuq
#SBATCH --ntasks-per-node=4
#SBATCH --gpus-per-node=4
#SBATCH --cpus-per-task=8
IMG=nvcr.io/nvidia/pytorch:26.09-py3
MOUNTS=$PWD:/workspace
srun --container-image=$IMG --container-mounts=$MOUNTS --container-workdir=/workspace python treino_ddp.py
```

Comparando as abordagens

	Apptainer	**Pyxis + Enroot**	`--container` **(OCI)**
Quem integra	você chama o binário	plugin SPANK no `srun`	o próprio Slurm (≥ 21.08)
Formato	`.sif`	squashfs / registry	bundle OCI
Sintaxe	`srun apptainer exec img.sif cmd`	`#SBATCH --container-image=img`	`srun --container $BUNDLE cmd`
GPU	flag do Apptainer	automático via Enroot	depende do runtime
MPI	host MPI + container	`srun --mpi=pmix`	depende do runtime

Capítulo 7

Boas práticas e encerramento

Peça o que vai usar

- **Tempo**: estimativa realista + margem — jobs curtos entram em "buracos" do escalonador (*backfill*)
- **Memória**: baseie-se no `MaxRSS` de execuções anteriores
- **CPUs**: confira `TotalCPU` × `CPUTime`; código serial não acelera com `-c 32`
- **Nós inteiros** quando o site exige
- **GPUs**: só quando o código realmente usa; CPUs proporcionais

Organize o trabalho

- Muitas tarefas parecidas → **job array** (com `%` limitando concorrência)
- Muitas tarefas pequenas → **vários steps** em um job, não milhares de jobs
- Pipelines → **dependências** (`afterok` , `aftercorr` , `singleton`)
- Teste em partição de **teste/devel** antes de escalar
- `sbatch --test-only` para validar e estimar início

Ambiente reprodutível

- Carregue **módulos dentro do script**, com versão explícita (`module load <pacote>/<versão>`)
- **Não** misture toolchains de versões diferentes
- **Não** mexa em `LD_LIBRARY_PATH` à mão
- Compile no **login node da mesma arquitetura** dos nós de destino
- Containers (Apptainer/Pyxis) para pilhas complexas

Dados e E/S

- O Slurm **não copia** seus arquivos: tudo precisa estar em FS **compartilhado** (ou use `sbcast`)
- O `/tmp` local é **compartilhado entre os jobs** do nó — limpe ao terminar
- Muitos arquivos pequenos em array grande → cuidado com o FS paralelo
- Crie diretórios de log **antes** de submeter

```
salloc -N1024 bash
sbcast a.out /tmp/joe.a.out
srun /tmp/joe.a.out
srun rm /tmp/joe.a.out
```

- Prefira usar o `/work` ao invés do `/tmp`.

Para saber mais

- Documentação oficial: https://slurm.schedmd.com/documentation.html
- Quick Start do usuário: https://slurm.schedmd.com/quickstart.html
- Resumo de comandos (PDF): https://slurm.schedmd.com/pdfs/summary.pdf
- Job arrays: https://slurm.schedmd.com/job_array.html
- MPI: https://slurm.schedmd.com/mpi_guide.html
- GPUs/GRES: https://slurm.schedmd.com/gres.html
- Containers: https://slurm.schedmd.com/containers.html
- Pyxis: https://github.com/NVIDIA/pyxis

Obrigado!

Perguntas?

A seguir: Laboratório

Laboratório

Mãos na massa

Os scripts e o roteiro completo estão no GitHub:

github.com/aveleda/rnp-palestra-261022

No cluster, uma cópia está em `/tmp/palestra/lab`



Antes de começar

```
$ ssh tng@<servidor>
tng@headnod/e-df-001:~$ mkdir <escolha um nome de diretorio>
tng@headnode-df-001:~$ cd <nome do diretorio criado>
tng@headnode-df-001:~$ sinfo -s                        # escolha a partição gpuq
tng@headnode-df-001:~$ mkdir -p logs                   # necessário para alguns exercícios
tng@headnode-df-001:~$ cp -p /tmp/palestra/lab/* .     # copiar os arquivos do lab
```

> Opções da **linha de comando** prevalecem sobre as do script.

Bom laboratório!

`README.md` tem o roteiro completo, com perguntas para cada exercício.